

Python - Lists

Python Lists

List is one of the built-in data types in Python. A Python list is a sequence of comma separated items, enclosed in square brackets []. The items in a Python list need not be of the same data type.

Following are some examples of Python lists –

```
list1 = ["Rohan", "Physics", 21, 69.75]
list2 = [1, 2, 3, 4, 5]
list3 = ["a", "b", "c", "d"]
list4 = [25.50, True, -55, 1+2j]
```

List is an ordered collection of items. Each item in a list has a unique position index, starting from 0.

A list in Python is similar to an array in C, C++ or Java. However, the major difference is that in C/C++/Java, the array elements must be of same type. On the other hand, Python lists may have objects of different data types.

A Python list is mutable. Any item from the list can be accessed using its index, and can be modified. One or more objects from the list can be removed or added. A list may have same item at more than one index positions.

Accessing Values in Lists

To **access values in lists**, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example –

```
list1 = ['physics', 'chemistry', 1997, 2000];
list2 = [1, 2, 3, 4, 5, 6, 7 ];
print ("list1[0]: ", list1[0])
print ("list2[1:5]: ", list2[1:5])
```

When the above code is executed, it produces the following result –

```
list1[0]:  physics
list2[1:5]:  [2, 3, 4, 5]
```

Updating Lists

You can update single or multiple elements of lists by giving the slice on the left-hand side of the assignment operator, and you can add to elements in a list with the `append()` method. For example –

```
list = ['physics', 'chemistry', 1997, 2000];
print ("Value available at index 2 : ")
print (list[2])
list[2] = 2001;
print ("New value available at index 2 : ")
print (list[2])
```

When the above code is executed, it produces the following result –

```
Value available at index 2 :
1997
New value available at index 2 :
2001
```

Delete List Elements

To **remove a list element**, you can use either the `del` statement if you know exactly which element(s) you are deleting or the `remove()` method if you do not know. For example –

```
list1 = ['physics', 'chemistry', 1997, 2000];
print (list1)
del list1[2];
print ("After deleting value at index 2 : ")
print (list1)
```

When the above code is executed, it produces following result –

```
['physics', 'chemistry', 1997, 2000]
After deleting value at index 2 :
['physics', 'chemistry', 2000]
```

Note – remove() method is discussed in subsequent section.

Python List Operations

In Python, List is a sequence. Hence, we can concatenate two lists with "+" operator and concatenate multiple copies of a list with "*" operator. The membership operators "in" and "not in" work with list object.

Python Expression	Results	Description
[1, 2, 3] + [4, 5, 6]	[1, 2, 3, 4, 5, 6]	Concatenation
['Hi!'] * 4	['Hi!', 'Hi!', 'Hi!', 'Hi!']	Repetition
3 in [1, 2, 3]	True	Membership

Indexing, Slicing, and Matrixes

Because lists are sequences, indexing and slicing work the same way for lists as they do for strings.

Assuming following input –

```
L = ['spam', 'Spam', 'SPAM!']
```

Python Expression	Results	Description
-------------------	---------	-------------

L[2]	SPAM!	Offsets start at zero
L[-2]	Spam	Negative: count from the right
L[1:]	['Spam', 'SPAM!']	Slicing fetches sections

Python List Methods

Python includes following list methods –

Sr.No.	Methods with Description
1	list.append(obj) Appends object obj to list.
2	list.clear() Clears the contents of list.
3	list.copy() Returns a copy of the list object.
4	list.count(obj) Returns count of how many times obj occurs in list
5	list.extend(seq) Appends the contents of seq to list
6	list.index(obj) Returns the lowest index in list that obj appears
7	list.insert(index, obj) Inserts object obj into list at offset index
8	list.pop(obj=list[-1]) Removes and returns last object or obj from list
9	list.remove(obj) Removes object obj from list
10	list.reverse() Reverses objects of list in place
11	list.sort([func]) Sorts objects of list, use compare func if given

Built-in Functions with Lists

Following are the built-in functions we can use with lists –

Sr.No.	Function with Description
1	cmp(list1, list2) Compares elements of both lists.
2	len(list) Gives the total length of the list.
3	max(list) Returns item from the list with max value.
4	min(list) Returns item from the list with min value.
5	list(seq) Converts a tuple into list.

TOP TUTORIALS

[Python Tutorial](#)
[Java Tutorial](#)
[C++ Tutorial](#)
[C Programming Tutorial](#)
[C# Tutorial](#)
[PHP Tutorial](#)
[R Tutorial](#)
[HTML Tutorial](#)
[CSS Tutorial](#)
[JavaScript Tutorial](#)
[SQL Tutorial](#)

TRENDING TECHNOLOGIES

[Cloud Computing Tutorial](#)
[Amazon Web Services Tutorial](#)
[Microsoft Azure Tutorial](#)
[Git Tutorial](#)
[Ethical Hacking Tutorial](#)

Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification



Chapters ▾

Categories ≡

Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler
Online Bash Terminal
Online SQL Compiler
Online Html Editor

ABOUT US | OUR TEAM | CAREERS | JOBS | CONTACT US | TERMS OF USE |
PRIVACY POLICY | REFUND POLICY | COOKIES POLICY | FAQ'S



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.